The GIL, Not the Syscall: What Batching Buys a Python `io_uring` Runtime on Multiple Cores

Marimuthu Velayutham*

October 7, 2026

Abstract

Linux `io_uring` is finding its way into Python event loops, and where it makes them faster than `asyncio` the gain is put down to fewer system calls. We built one small server six ways to find out what batching buys when worker threads share one CPython interpreter. The answer is the Global Interpreter Lock (GIL). A Python loop on `epoll` that releases the GIL around every system call slows down as threads are added: four threads on four cores deliver 0.64 times the throughput of one. With the same two system calls per request but one GIL release per pass over the ready sockets, the same loop scales by **??S:factorial/g/py-epoll-batch/thread/4??**×, about as well as a Python loop on `io_uring` (2.21×) and nearly as well as loops written in C (2.34×). Batching helps threads mainly because the GIL changes hands less often; the saved system calls matter much less. We describe `uringpy`, a runtime whose workers reap completions in C and take the GIL once per batch of requests. On a standard CPython 3.14 build its threads keep pace with separate processes when no Python runs per request (328.4 thousand requests per second), and reach 0.93 of process throughput with a short Python handler, against 0.67 when the GIL is taken per request. We also show where this stops working: handlers that hold the GIL for most of a request, the free-threaded build, and large responses. Code, raw measurements and the scripts that produce every table, figure and number are public.

Index Terms— io_uring, asynchronous I/O, Python, Global Interpreter Lock, event loop, multicore scaling, system-call amortization

1 Introduction

Linux `io_uring` [1] lets a program queue many I/O operations and collect many results with a single system call. With `epoll`, by contrast, the kernel only reports which sockets are ready, and the program still makes one system call for each read and each write. Python projects have started to put `io_uring` underneath an `asyncio`-compatible event loop [2, 3]. Where such a loop is reported to be faster than the standard one, the gain is credited to taking system calls out of the hot path [3].

That explanation is plausible, but it is hard to check from an end-to-end benchmark, because three things change at once when one event loop is swapped for another. The number of system calls changes. So does the amount of work done per event, since some loops dispatch each event in C and others in Python. And so does the way the loop treats CPython's Global Interpreter Lock (GIL), which allows only one thread to run Python code at a time. Every time a worker thread gives the lock up it has to get it back, and when other threads want the lock too, getting it back can mean being put to sleep and woken again. A typical `epoll` loop written in Python makes more system calls, runs more bytecode and releases the GIL more often than a C loop on `io_uring`, so comparing the two cannot tell us which of these differences is responsible.

In this paper we separate them. We wrote the same minimal server loop four times, on `io_uring` and on `epoll`, once in C with the GIL released and once in Python. We then added two more loops that each differ from one of those four in a single detail. The first is the Python `epoll` loop with its system calls left exactly as they were, except that they are issued in a way that releases the GIL once per pass over the ready sockets and not once per call. The second is the Python `io_uring` loop modified to hold on to the GIL while it submits.

With a single worker, both the interface and the language of the loop make a modest difference. `io_uring` is 1.14–1.25 times faster than `epoll`, and a C loop is 1.05–1.15 times faster than the equivalent Python loop. With several worker threads in one interpreter the picture changes, and what matters is how often the GIL changes hands. The Python loop on `epoll` gives the lock up 2.012 times per request, and four threads of it manage only 0.64 times the throughput of one. Keeping the interface and the **??sys:factorial/g/py-epoll-batch/thread/4??** system calls per request, but cutting the releases to **??ho:factorial/g/py-epoll-batch/thread/4??** per request, lifts that to **??S:factorial/g/py-epoll-batch/thread/4??**. This is about the scaling we measure for the Python loop on `io_uring` (2.21), and it is close to the two C loops (2.34 and 2.41). So batching does help a multi-threaded

*M. Velayutham is an independent researcher (e-mail: marimuthu.vm@gmail.com; ORCID: 0009-0007-7352-5552).

Python server, but mostly because it reduces GIL hand-offs. The saved system calls are a smaller effect, and a completion-based interface is one convenient way of getting the batching, not a requirement for it.

We used this observation to build `uringpy`, a small runtime that reaps completions in C. It keeps hand-offs rare in three ways. Its basic reactor runs the whole accept, receive and send cycle inside a single `nogil` region and never executes Python. A second mode calls a Python handler for each request but takes the GIL once for a whole batch of requests. Finally, it exposes a batch interface on which a loop can be written in Python. Each worker has its own ring and its own `SO_REUSEPORT` listener, and N workers scale across cores inside one interpreter on an ordinary GIL build. We also looked for the limits of the approach and found four: the free-threaded build, where `asyncio` threads scale without help; handlers long enough that the lock is simply busy; light load, where there is little to batch; and large responses, where the network becomes the bottleneck.

The paper makes three contributions.

- A cost model for worker threads under the GIL with two terms that we measure directly, namely how long a request holds the lock and how many hand-offs it causes, together with experiments that vary the system-call interface, the placement of the loop and the number of hand-offs one at a time (Sections 3 and 5).
- `uringpy`, a runtime that reaps completions and runs the connection protocol in C, and that takes the GIL once per batch when it calls a Python handler. With a short handler this raises thread scaling on four cores from 1.55× to 2.12× (Section 4).
- An evaluation that covers threads and processes on both the GIL and the free-threaded build, with and without a Python handler, across connection counts and response sizes. We report confidence intervals and processor usage throughout, and we include the results that went against us (Section 5).

The rest of the paper is organised as follows. Section 2 reviews related work. Section 3 develops the cost model, and Section 4 describes the runtime. Section 5 presents the experiments and their results. Section 6 discusses what they imply, Section 7 lists the threats to validity, and Section 8 concludes.

2 Background and Related Work

Reducing system calls. The cost of entering the kernel for every I/O operation has been studied for a long time. FlexSC [4] showed that issuing system calls in batches and running them asynchronously removes much of the overhead of switching modes, and MegaPipe [5] combined batching with per-core listening sockets. `io_uring` [1] brought the idea into mainline Linux through a pair of rings shared between the application and the kernel. Most measurements of `io_uring` so far concern storage. Didona et al. [6] compared it with `libaio` and SPDK and found that its performance depends heavily on how it is configured. Jasny et al. [7] found that dropping it into an existing database system brings little, while designing around its asynchronous, batched model brings a lot. Both studies caution against crediting a speed-up to the interface alone. We ask the same question for a network server in a managed language, where an interpreter lock is involved in handling every event.

The GIL. CPython runs bytecode under a global lock. Beazley [8] described how the lock is passed between threads and how badly I/O-bound threads can fare when others compete for it, which is the behaviour at the centre of this paper. Two ways around the lock are relevant here. One is to remove it: the free-threaded build [9] is officially supported, though not the default, from Python 3.14 [10], and the standard event loop has been reworked to scale on it [11]. This is why we evaluate on CPython 3.14 both with and without the GIL. The other is to move work into native code that releases the lock, which Cython [12] makes straightforward with `nogil` blocks, and which is what we do. Whatever still runs under the lock limits scaling in the way Amdahl described [13], and Eyerman and Eeckhout [14] extended that argument to contended critical sections. Section 3 adapts both to the GIL.

That contention for the GIL hurts I/O-bound threads has therefore been known for some time. What has been missing is a measurement that ties it to the design of an event loop: how many hand-offs a given loop causes per request, what each one costs, and how much scaling comes back when they are batched. This paper supplies those measurements and uses them to separate the effect of the lock from that of the system-call interface.

Python event loops and servers. `uvloop` [15] replaces the `asyncio` loop with one built on libuv and is the usual fast baseline. `uringloop` [2] and `uringcore` [3] are `asyncio` loops backed by `io_uring`. The second reports gains over `asyncio` and attributes them to its completion-driven design, which removes system calls from the hot path; the first makes no performance claim. Both deliver events to Python callbacks, as any `asyncio` loop must. Doing network I/O outside the interpreter is not new either. Granian [16], for instance, runs it on Rust threads and enters Python only for the application. Our runtime belongs to the same family, but it is deliberately small, so that we could pair it with an otherwise identical `epoll` reactor and with Python loops on both interfaces. That pairing is what lets us separate the interface, the placement of the loop and the GIL hand-offs. We did not benchmark the `io_uring` loops cited above and make no claim about how any of them behaves. Our workers follow the usual shared-nothing layout, with one event loop and one `SO_REUSEPORT` listener [17] per worker.

Measurement. Benchmarks are easy to get wrong when runs are few or the setup is uncontrolled [18]. We

Table 1: Notation.

N	number of workers (threads of one interpreter, or processes)
X_N, $\bar{x}_N$	throughput of N workers in requests per second: in the model, and as the mean of the measured runs
S_N	scaling, $\bar{x}_N/\bar{x}_1$
t_p	processor time per request spent holding the GIL
t_c	processor time per request spent with the GIL released
f	share of a request spent holding the GIL, $t_p/(t_c+t_p)$
g	GIL hand-offs per request: times the lock is released or acquired on behalf of one request
τ	cost of one hand-off when another thread is waiting
w	time a request waits to acquire the GIL
U_N	GIL utilisation, $\bar{x}_N\, t_p$: share of each second the lock is held running request code
C_N	busy server cores; E_N: throughput per busy core relative to one worker

repeat every configuration, interleave the repetitions, and report means with confidence intervals alongside processor usage on both machines. Load comes from `wrk` [19], which is a closed-loop generator: it waits for each response before sending the next request on a connection, so its latency figures understate what an open-loop client would see. We use them only to compare configurations under the same load.

3 A Cost Model for Worker Threads under the GIL

Table 1 lists the symbols used in the rest of the paper.

A bound from time under the lock. Suppose a request needs t_c of processor time that does not touch interpreter state, and can therefore run with the GIL released, plus t_p that has to run with the GIL held. One saturated worker then serves $X_1 = 1/(t_c + t_p)$ requests per second. Now run N worker threads in one interpreter on at least N cores. Each worker still spends $t_c + t_p$ on every request, so $X_N \le N/(t_c+t_p)$. In addition, only one thread can hold the GIL at any moment, so all requests together can occupy it for at most one second per second: $X_N\, t_p \le 1$. Dividing by X_1 gives

$$S(N) = \frac{X_N}{X_1} \le \min\left(N,\ \frac{1}{f}\right), \qquad f = \frac{t_p}{t_c + t_p}, \qquad (1)$$

where f is the fraction of a request's processor time spent holding the GIL. This is Amdahl's law [13] with the GIL-held portion playing the serial part. Two extreme cases are worth keeping in mind. A loop that holds the GIL for everything it does has $f = 1$ and can never beat a single worker, however many threads it runs. A loop that never enters the interpreter has $f = 0$ and is limited only by N. A loop written in Python can sit anywhere in between, because the system calls it makes with the GIL released count towards t_c. When each worker is its own process, each has its own GIL, the second constraint disappears, and $S(N) \le N$ for any f. We therefore treat process-mode results as the reference for what the hardware can deliver.

The cost of handing the lock over. Equation (1) only accounts for the time the GIL is held. Passing it from one thread to another is not free: the thread that releases it has to wake a waiter, and that waiter has to be scheduled before it can do anything. Eyerman and Eeckhout model costs of this kind for critical sections in general [14]. Say a request causes g hand-offs, each costing τ when another thread is waiting. The GIL is then tied up for $X_N\,(t_p + g\,\tau)$ of every second, which gives

$$X_N \le \frac{1}{t_p + g\,\tau}, \qquad S(N) \le \min\left(N,\ \frac{1}{f + g\,\tau\,X_1}\right). \qquad (2)$$

The count g depends on how the server is built and not on the request. If a loop releases the GIL around every system call, g equals its system calls per request. A C reactor that takes the GIL for each handler call has $g = 1$. A loop that gives the GIL up once per batch of B requests has g around $1/B$. This is the only place where the system-call interface comes into the model, and it does so indirectly. A completion interface hands back many completions per call, so a loop built on it naturally releases the GIL once per batch. A readiness loop written in the usual way releases it at every read and every write. Nothing forces that, though. A readiness loop can batch its releases too, and one of the loops in Section 5 does exactly this.

We should be clear about how much we expect of (2). It is a bound, and it predicts a direction; it is not a calibrated formula. The cost τ is not a constant. It depends on whether anyone is waiting, on how quickly the kernel schedules the waiter and on the interpreter's switch interval, and in Section 5 we find that no single value fits all our configurations. What the model does predict, and what the experiments test, is an ordering. If f stays the same, scaling should improve as g falls, whatever the interface and whatever language the loop is written in. And once g is small, scaling should be limited by $1/f$.

What we measure. We measure these quantities and do not assume them. We obtain g by counting GIL releases in the Python loops and GIL acquisitions in the C reactors. We time every handler call to get t_p and the time w that a request waits for the GIL. We record the voluntary context switches of the server process, which go up when threads sleep waiting for the lock. From these we compute the *GIL utilisation* $U_N = \bar{x}_N\, t_p \le 1$, the share of each second in which the GIL is held running request code. The remainder, $1 - U_N$, is either spent on hand-offs or idle.

A probe without I/O. The two extremes of (1) can be demonstrated without a server at all. We process a fixed number of synthetic events in two ways, using the same arithmetic per event: in an interpreted loop ($f = 1$) and inside a single C `nogil` region ($f = 0$). Meanwhile K CPU-bound Python threads compete for the GIL. With one competitor the interpreted loop takes 2.04 times as

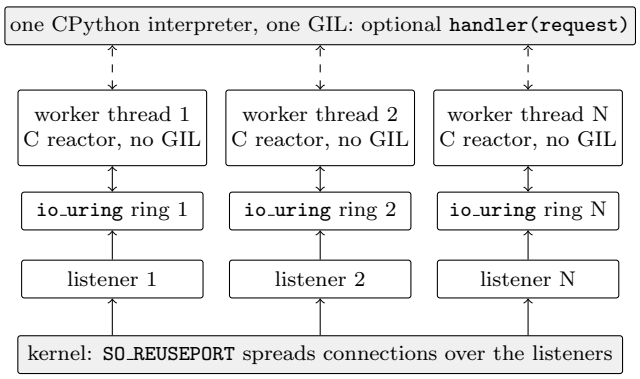


Figure 1: A server is N engines in one process. Solid arrows are the data path, which never leaves C; the dashed arrow is the only point where a worker takes the GIL, and exists only in application mode.

long as it does alone, while the C loop takes 1.02 times as long (20 repetitions). In other words, a thread holding the GIL shares it about evenly with a competitor, and a thread that has let go of it is unaffected. With two or three competitors the timings on our four-core machine vary too much to be useful for either loop, so we do not draw on them; the full output is in the repository. Since the probe does no I/O and makes no system calls, it tells us nothing about the interface or about hand-offs. For those we need the server experiments of Section 5.

4 Design

`uringpy` is a Cython [12] extension built on `liburing`. Its basic unit is the *engine*, which consists of one `io_uring` instance, a pool of fixed-size receive buffers and a reactor loop. A server is made of N engines. Each runs on its own OS thread and has its own `SO_REUSEPORT` listening socket on the shared port, so the kernel distributes incoming connections among the workers (Fig. 1). The engines share nothing in user space. No ring, buffer or connection is ever touched by two workers, and so the data path needs no user-level locks. Rings are created with default flags, and we use plain single-shot accept, receive and send operations without registered buffers or files. The interface effects we report in Section 5 therefore apply to this basic way of using `io_uring`.

The loop. Fig. 2 shows the loop that each worker runs, all of it inside one `nogil` region. Every iteration makes exactly one system call, `io_uring_submit_and_wait`. That call passes to the kernel whatever operations were queued while the previous batch was being handled, and returns once at least one completion is available. The loop then works through all the completions already sitting in the ring without going back to the kernel. Any follow-up operations, such as the next receive or the send of a response, are written to the submission ring and go out with the next iteration's call. No Python object is created for a completion, and the interpreter is never entered.

```
1:  queue ACCEPT on the listener
2:  while not stopped do
3:    io_uring_submit_and_wait(1) {one system call}
4:    for each completion q in the ring, up to the batch
      cap do
5:      if q is an ACCEPT then
6:        queue ACCEPT; queue RECV on the new con-
          nection
7:      else if q is a RECV of n > 0 bytes then
8:        if application mode then
9:          take GIL; r ← handler(request)
10:         copy r to the connection's buffer; release
            GIL
11:       end if
12:       queue SEND of the response from offset 0
13:     else if q is a SEND of n > 0 bytes then
14:       if part of the response is still unsent then
15:         queue SEND of the remainder
16:       else
17:         count one request; queue RECV
18:       end if
19:     else
20:       close the connection {peer closed, or error}
21:     end if
22:   end for
23: end while
```

Figure 2: The reactor loop of one worker. Queued operations reach the kernel at the next execution of line 3. With GIL batching, lines 9–10 are deferred: the requests of a pass are collected and the GIL is taken once for all of them.

Per-operation state. Each operation carries a 64-bit tag, which the kernel returns with the completion. Eight bits give the operation type, 32 the file descriptor, and the remaining 24 hold either the receive-buffer slot (for a receive) or the number of response bytes already sent (for a send). Dispatch needs no lookup table as a result. Because the offset travels with the operation, a send that the kernel completes only partially can be resumed where it stopped, and a response may be larger than a buffer slot (up to 16 MiB). Receive buffers come from a per-engine slab allocator. If no buffer or submission entry is available for a connection, the connection is closed instead of being left hanging.

Two entry points. `serve_forever_echo` replies to every request with a fixed response and never takes the GIL, so it measures the transport path by itself. `serve_forever_app` hands each request to a Python callable. The reactor takes the GIL only for that call. It builds the request `bytes`, invokes the handler, copies the result into a per-connection buffer that the engine owns, and releases the GIL before queueing the send. Accept, receive and send stay outside the GIL in both modes.

By default the GIL is taken once per request, which

means one hand-off per request whenever another worker is waiting ($g = 1$ in (2)). With *GIL batching* switched on, the reactor instead collects all the requests it finds in one pass over the completion queue and takes the GIL once for the lot. The handler sees the same calls in the same order. The difference is that the lock changes hands once per batch, and the cost is that no response from a batch is queued until every handler in it has finished. An optional cap limits how many requests are handled under one acquisition. This bounds the delay, and it also lets us vary g between 1 and the reciprocal of the batch size in the experiments. If a handler raises an exception, that connection is closed and the error is counted. An interrupt such as `KeyboardInterrupt` or `SystemExit` stops the reactor and is re-raised in the thread that started it. Neither entry point parses HTTP. Each replies once per `recv`, so pipelined and fragmented requests are out of scope, and the same is true of every baseline we compare against.

A batch interface for Python. The engine also makes submission and reaping available to Python code. Operations are queued without a system call, `flush` submits everything that is queued, and `get_events` returns the completions that have arrived as a list. Both calls release the GIL around their system call, so a Python loop written on this interface gives the GIL up twice per batch no matter how many requests the batch holds. One detail turns out to matter a great deal. The kernel may carry out queued operations, a send for example, inside the submitting call itself. Whether the GIL is held across that call therefore determines how much of a request's processor time is spent under the lock.

Instrumentation. The reactor counts its `io_uring_enter` calls, its completions and the responses it has fully sent, which means system calls per request and completions per system call are measured and not estimated. A batch cap limits the completions processed per system call so that batching can be varied by itself. GIL acquisitions for handler calls are counted, and optional timing records how long each handler call waited for the GIL and how long it held it. The counters are plain integer increments, and the timing is off unless requested.

An `epoll` twin. For the comparisons in Section 5 the package also includes `EpollEngine`, which is the same loop written for `epoll`. It makes one `recv` and one `send` system call per request, replies once per `recv`, resumes partial sends, runs inside one `nogil` region and counts every system call. It offers the same two entry points and the same GIL batching: in application mode it reads from every ready socket, takes the GIL once to run the handlers for that pass, and then sends the responses. There is also a small helper, `BatchIO`, for Python loops on `epoll`. The loop passes it the sockets that `epoll` reported as ready, and a single call then performs the `recv` and `send` for each of them with the GIL released the whole time. These are the same system calls the loop would have made through CPython's socket objects. All that changes is how often the GIL is released, which drops from once per call to once per pass.

Table 2: Testbed.

Server	Google Compute Engine `t2d-standard-4`: 4 vCPUs (AMD EPYC 7B13), one per physical core, 16 GB
Client	`t2d-standard-8`: 8 vCPUs (AMD EPYC 7B13), one per physical core; same zone (`us-east1-b`)
System	Ubuntu 24.04 LTS, Linux 7.0.0-1011-gcp on both
Runtime	Docker, Debian 12 base, `liburing` 2.6; CPython 3.14.7 built from source with and without `--disable-gil`
Load	`wrk` 4.1.0, 8 threads, 400 keep-alive connections unless stated, `GET /`
Response	13-byte body on the transport path; 19-byte JSON body built by the handler; other sizes where stated
Run	fresh server process; 5 s warm-up (discarded), 20 s measured
Repetitions	5 per configuration, interleaved in shuffled order

Interpreter integration. When a signal interrupts a wait, control returns to the interpreter for long enough to run any pending Python signal handlers, so a worker on the main thread can be stopped in the normal way. The extension declares itself safe for free-threaded builds, which means loading it there does not switch the GIL back on. The benchmark image checks this when it is built.

5 Evaluation

5.1 Setup

Table 2 describes the testbed. The server and the load generator are separate virtual machines in the same zone. Both belong to a machine family in which every vCPU is a full physical core [20], so the number of workers is also the number of cores in use. The client has twice as many cores as the server so that it does not become the bottleneck. The code under test runs in a container on the server with host networking. Both interpreters are built from one recipe that differs only in the `--disable-gil` flag. Comparing them is therefore not also a comparison of compilers or build options, and every run records the interpreter version and the GIL state reported by the server process.

All results in this paper were measured at a single commit of the code (`5b28fd1, b60fe24, d9bc686`) by a single script. The script sends that commit to the server, builds both images from it, runs the test suite inside each image and then runs the experiments one after another. Conditions that could differ between engines are set in one place. In particular, every listening socket has a backlog of 1024 and `TCP_NODELAY` set, and accepted connections inherit the latter.

Table 3: Engines. "C" means the loop runs with the GIL released; the handler is the same Python function for every engine that calls one.

Engine	Interface	Per-event loop and GIL
`uringpy`	`io_uring`	C
`c-epoll`	`epoll`	C (same loop)
`py-uring`	`io_uring`	Python; GIL released per pass
`py-epoll`	`epoll`	Python; GIL released per syscall
`py-epoll-batch`	`epoll`	as `py-epoll`, GIL released per pass
`py-uring-held`	`io_uring`	as `py-uring`, GIL held in submit
`asyncio`	`epoll`	Python, streams API
`asyncio-proto`	`epoll`	Python, Protocol API
`uvloop(-proto)`	libuv	C loop, Python callbacks
`uringpy-app`	`io_uring`	C + handler, GIL per request
`uringpy-app-batch`	`io_uring`	C + handler, GIL per batch
`c-epoll-app`	`epoll`	C + handler, GIL per request
`c-epoll-app-batch`	`epoll`	C + handler, GIL per batch
`asyncio-app`	`epoll`	Python, streams API + handler
`asyncio-proto-app`	`epoll`	Python, Protocol API + handler
`uvloop-proto-app`	libuv	C loop, Protocol API + handler

5.2 Engines

Table 3 lists the servers we compare. They all answer each `recv` with the same bytes and none of them parses HTTP, so the only differences are in how the I/O loop is built and when the GIL is held. Every engine runs as N workers with one `SO_REUSEPORT` listener each, either as threads of one interpreter (*thread* mode, one GIL) or as separate processes (*process* mode, one GIL per worker).

The first four engines make up a 2×2 design that varies the system-call interface and the placement of the per-event loop. The two Python loops in it are bare dispatch loops written directly against the interface. They have none of `asyncio`'s tasks, futures and transports, and nobody would deploy them as they are. The design has a confound, and it turns out to be the thing that matters most. `py-epoll` makes its `recv` and `send` calls through CPython's socket objects, which release the GIL around every system call. `py-uring` uses the batch interface of Section 4 and releases the GIL around its two calls per pass. Going from one cell to the other therefore changes the interface, the number of system calls and the number of GIL releases all at once.

We added two controls to pull these apart. Each differs from a cell of the design in exactly one respect, and neither is a cell itself. `py-epoll-batch` is `py-epoll` with the `recv` and `send` calls of each pass issued through `BatchIO`, which keeps the interface and the system calls but releases the GIL once per pass for all of them. `py-uring-held` is `py-uring` changed to keep the GIL while it submits. Since the kernel performs the queued sends inside that call, the GIL is released just as rarely, but most of a request's processor time now falls under the lock. In terms of the model, the first control changes g without touching the interface, and the second changes f without touching g.

5.3 Metrics

For a configuration with n runs and throughputs $x_1, \ldots, x_n$ (requests per second over the measured interval) we report the mean $\bar{x}$ together with the half-width e of its 95% confidence interval,

$$e = t_{0.975,\,n-1}\,\frac{s}{\sqrt{n}}, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i-\bar{x})^2, \tag{3}$$

where t is the Student-t quantile (2.776 for $n = 5$). Scaling at N workers is the ratio to the one-worker configuration of the same engine and mode, and its half-width follows from first-order error propagation:

$$S_N = \frac{\bar{x}_N}{\bar{x}_1}, \qquad e_S = t\,S_N\sqrt{\frac{s_N^2}{n_N\,\bar{x}_N^2}+\frac{s_1^2}{n_1\,\bar{x}_1^2}}, \tag{4}$$

with t taken at $\min(n_N, n_1) - 1$ degrees of freedom. Any other ratio of two configurations is computed and reported in the same way.

We sample processor usage from `/proc/stat` on both machines immediately before and after each measured interval. If u is the busy share of the server's four cores, then $C = 4u$ is the number of busy cores. We report throughput per busy core, $\bar{x}/C$, next to S_N, because even a one-worker server uses other cores for kernel network processing and S_N by itself understates per-core scaling. We also give this quantity relative to one worker, $E_N = (\bar{x}_N/C_N)/(\bar{x}_1/C_1)$, so that $S_N = E_N\,C_N/C_1$. A configuration is flagged if the client's busy share exceeds 85%, since the load generator may then be the limiting factor.

The remaining metrics come from the servers' own counters, summed over a configuration's runs and then divided. They are system calls per request, completions per `io_uring_enter`, and GIL hand-offs per request g. The last is the counted number of GIL releases for the Python loops and the counted number of GIL acquisitions for the C reactors that call a handler. In application mode with timing enabled we also report the GIL hold time t_p and wait time w per handler call. Finally, every server process reports its voluntary context switches from `getrusage`. A thread that goes to sleep waiting for the GIL, or for I/O, counts as one. For our own engines we divide this by the number of requests the server counted. `asyncio` and `uvloop` do not count requests, so for them we divide by the requests that `wrk` counted, scaled from the measured interval to the server's lifetime. Where both methods can be applied they agree to within **??meta:csw-method-diff??**%. Latency percentiles are taken from `wrk` and are used only to compare configurations under the same load (Section 2).

5.4 How the hypothesis changed

The claim we test in this paper is not the one we started with. Since that affects how each result should be read, we describe what happened in the order it happened.

We began with a broader hypothesis: for a Python `io_uring` server, where the per-event loop runs, in C or in Python, matters more than which system-call interface it uses. We wrote down a criterion for it on the day of the first measurements, while they were already running, so it cannot count as pre-registered. The hypothesis would be supported if moving the loop from Python to C changed throughput far more than switching between `epoll` and `io_uring` did, and if capping the completions processed per system call changed throughput only slightly.

The first measurements appeared to support part of this. The Python loop on `io_uring` did not scale across threads, and we concluded that a Python loop could not scale on either interface. That conclusion was wrong. Our loop was holding the GIL during its submitting system call, which nothing in the design required. Once the GIL was released around that call, the same loop scaled. We have kept the faulty variant as the control `py-uring-held`, and the earlier runs remain in the repository, marked as superseded.

The corrected 2×2 design contradicts the broad hypothesis (Section 5.5). It pointed us instead to the narrower statement of Section 3, that thread scaling depends on how much of a request runs under the GIL and on how often the GIL changes hands. We arrived at this statement after looking at those data, so the 2×2 results are the observations that led to it and should not be read as a test of it.

To test it we designed three further experiments and ran them afterwards: the control `py-epoll-batch`, the `epoll` reactor with GIL batching, and a sweep over the number of requests served per GIL acquisition. For the first of these we recorded the prediction, and what would refute it, in the repository's description of the experiments before the runs, in the same commit that the runs were made at. If hand-offs decide thread scaling, `py-epoll-batch` should scale like `py-uring` even though it uses `epoll`; if it scales like `py-epoll`, the explanation is wrong. The other two experiments are listed there as open questions. All experiments were then re-run together at that commit, which is why the tables below show the 2×2 design re-measured alongside its controls.

5.5 Results

All tables and figures, and every measured number quoted in the text, are generated from the raw runs by a script in the repository (`benchmarks/make_tables.py`). Nothing is copied by hand. The data set consists of 1135 runs of 227 configurations, of which 0 failed. Hypervisor steal time never exceeded 0.0% on either machine. Averaged over the runs of a configuration, the client's busy share stayed at or below 77%, and 1 individual runs went above 85%. 0 runs reported a non-2xx response and 0 a handler error. 80 runs reported socket errors (experiments: handler), affecting at most 0.39% of a run's requests. These occur with the longest handlers, and we come back to them in Section 7.

The 2×2 design. Table 4 shows the four cells, and Table 5 gives the effect of each factor with the other held fixed. The counters confirm that the design varies what it is supposed to. At four workers the two `epoll` loops make 2.014 and 2.012 system calls per request, one `recv` and one `send`, while the two `io_uring` loops make 0.022 and 0.042.

On one worker both factors have an effect, and neither is dominant. Switching from `epoll` to `io_uring` raises throughput by a factor of 1.14 ± 0.04 for the C loop and 1.25 ± 0.03 for the Python loop. Moving the loop from Python to C raises it by 1.05 ± 0.04 on `io_uring` and by 1.15 ± 0.03 on `epoll`. A full event loop would pay more than this second pair suggests, because our Python loops are so bare. `asyncio`'s streams API, which also runs on `epoll`, reaches 66.8 thousand requests per second on one worker (Table 7), compared with 106.8 for `py-epoll`.

Four workers gave us a result we had not expected. Both C loops scale, to $S_4 = 2.34 \pm 0.07$ on `io_uring` and 2.41 ± 0.06 on `epoll`. But so does the Python loop on `io_uring`, with $S_4 = 2.21 \pm 0.04$ and 295.1 thousand requests per second. That is 0.90 of the C loop on the same interface and 0.99 of the C loop on `epoll`. The Python loop on `epoll` behaves quite differently. It gains a little at two workers (1.32 ± 0.03) and then drops below the throughput of a single worker at four (0.64 ± 0.01). So the placement of the loop is not what decides scaling. On `io_uring`, moving the loop from Python to C is worth 1.05× on one worker and 1.11× on four. Taken at face value, the table says that the interface is what separates the two Python loops, by a factor of 1.25 on one worker and 4.33 on four. The saved system calls cannot be the reason, because `c-epoll` makes two per request and scales as well as anything. The design cannot tell us more than that, however, because of the confound described above: between `py-epoll` and `py-uring` the GIL releases change along with the interface.

The hand-off control. This is what `py-epoll-batch` is for. It uses `epoll` and makes **??sys:factorial/g/py-epoll-batch/thread/4??** system calls per request, the same as `py-epoll` (2.012). The only difference is that at four workers it releases the GIL **??ho:factorial/g/py-epoll-batch/thread/4??** times per request, where `py-epoll` releases it 2.012 times. With that one change, four threads scale by **??Spm:factorial/g/py-epoll-batch/thread/4??** in place of 0.64 ± 0.01. They deliver **??x:factorial/g/py-epoll-batch/thread/4??** thousand requests per second, which is **??rpm:handoff-epoll/4??** times what `py-epoll` manages and **??rpm:pybatch-over-pyuring/4??** of what `py-uring` does. Once the hand-offs are equal, the interface is worth **??rpm:iface-batched/1??** on one worker and **??rpm:iface-batched/4??** on four. The prediction we recorded in Section 5.4 could have failed here, and it did not. Whether these loops scale as threads is decided by the number of GIL hand-offs, and

Table 4: The 2×2 design and two controls (worker threads, GIL build, 13-byte body): throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling of worker threads S_4 and of worker processes S_4^{proc}; and, at four worker threads, per request: measured system calls (sys), GIL hand-offs (g) and voluntary context switches (c.sw.); busy server cores C_4; and 99th-percentile latency in ms.

Engine	Interface	Loop	N=1	N=2	N=4	S_4	S_4^{proc}	sys	g	c.sw.	C_4	p99$_4$
`uringpy`	`io_uring`	C	140.3 ± 4.3	230.6 ± 6.9	329.0 ± 0.8	2.34 ± 0.07	–	0.022	0	–	3.92	1.8
`c-epoll`	`epoll`	C	123.4 ± 2.9	200.5 ± 3.8	297.7 ± 0.8	2.41 ± 0.06	–	2.014	0	–	3.92	2.8
`py-uring`	`io_uring`	Python	133.5 ± 2.6	212.4 ± 2.9	295.1 ± 0.7	2.21 ± 0.04	–	0.042	0.042	–	3.81	1.9
`py-epoll`	`epoll`	Python	106.8 ± 1.8	141.6 ± 2.7	68.1 ± 0.9	0.64 ± 0.01	–	2.012	2.012	–	2.46	11.5
`py-uring-held`‡	`io_uring`	Python	131.5 ± 3.6	129.1 ± 2.8	128.6 ± 2.3	0.98 ± 0.03	–	0.040	–	–	1.51	783.7

The upper four rows are the design; the lower rows are controls that differ from one cell in one respect. ‡ `py-uring` holding the GIL during the submitting system call.

Table 5: Effect of each factor with the other held fixed, as a ratio of mean throughputs (± 95% half-width) from Table 4. Interface: `io_uring` over `epoll`. Loop: C over Python.

Effect	N=1	N=2	N=4
Interface, loop in C	1.14 ± 0.04	1.15 ± 0.04	1.11 ± <0.01
Interface, loop in Python	1.25 ± 0.03	1.50 ± 0.03	4.33 ± 0.06
Loop, on `io_uring`	1.05 ± 0.04	1.09 ± 0.04	1.11 ± <0.01
Loop, on `epoll`	1.15 ± 0.03	1.42 ± 0.04	4.37 ± 0.06

neither the interface nor the number of system calls has much to do with it.

The context switches show how this comes about. A single `py-epoll` worker makes **??csw:factorial/g/py-epoll/thread/1??** voluntary context switches per request. Four workers make **??csw:factorial/g/py-epoll/thread/4??**, roughly one for every release of the lock, which means that almost every time a worker lets go of the GIL it has to sleep before it gets it back. The batched loop makes **??csw:factorial/g/py-epoll-batch/thread/4??** per request and `py-uring` makes **??csw:factorial/g/py-uring/thread/4??**. The loops themselves are not at fault. Run as four processes, `py-epoll` scales by **??Spm:factorial/g/py-epoll/process/4??**, in line with the others (Table 4), so the loss comes from sharing the lock.

We should point out that this control is not perfectly clean. `BatchIO` also takes the `recv` and `send` calls out of CPython's socket objects, so `py-epoll-batch` runs less Python per request than `py-epoll` does. This is why it is **??rpm:handoff-epoll/1??** times faster on one worker, and it means the control lowers f a little as well as g. Even so, time under the lock cannot explain the gap at four workers. By (1) it would limit four threads to $1/f \geq 1$ times one worker, whereas `py-epoll` falls below one worker and sleeps once per release. That behaviour is what the hand-off term of (2) describes. For the same reason, if one wants to know what the interface is worth to a Python loop on one worker, the ratio of `py-uring` to `py-epoll-batch` (**??r:iface-batched/1??**) is a fairer estimate than the ratio of `py-uring` to `py-epoll` (1.25).

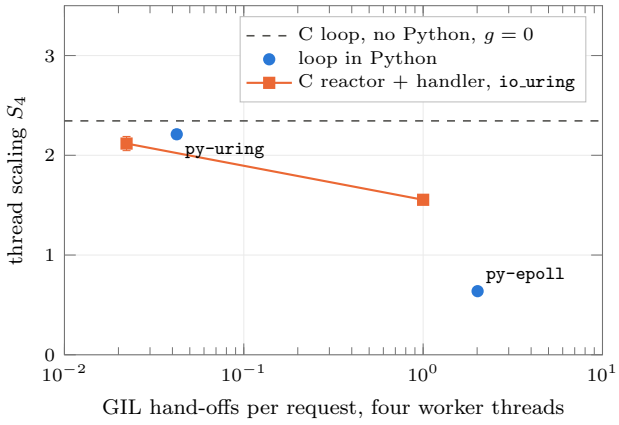


Figure 3: Thread scaling at four workers against measured GIL hand-offs per request (GIL build; bars are 95% half-widths). Loops in Python run the transport workload; the C reactors call the Python handler for every request, and the connected points are one reactor with the number of requests served per GIL acquisition varied. ‡ `py-uring-held` gives up the GIL as rarely as `py-uring` but holds it while the kernel performs the sends.

The hold-time control. Counting hand-offs does not tell the whole story, and (1) says it should not. `py-uring-held` gives up the GIL only **??ho:factorial/g/py-uring-held/thread/4??** times per request, which is less often than `py-uring`. It keeps the lock while submitting, though, and the kernel does the queued sends inside that call. Most of a request's processor time then falls under the GIL. Four workers end up delivering the throughput of one (0.98 ± 0.03), with only 1.51 of the four cores busy. `wrk` reports a 99th-percentile latency of 783.7 ms against a median of 0.9 ms, so some connections are served quickly while others starve. Releasing the GIL around that single call accounts for the whole difference between 0.98× and 2.21×. Fig. 3 puts all the thread-mode configurations for which we count hand-offs on one plot. Scaling falls as g rises, both for the Python loops and for the C reactors with a handler. The one point that sits far below the rest at small g is the loop whose f is close to 1.

Table 6: Batch cap (`uringpy`, one worker): completions processed per `io_uring_enter` limited to the cap. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), ratio to the uncapped run, measured completions per call and system calls per request.

Cap	Throughput	vs. none	compl./call	sys/req
1	115.3 ± 3.4	0.80 ± 0.03	1.0	2.001
4	134.2 ± 4.4	0.93 ± 0.04	4.0	0.501
16	143.3 ± 2.5	1.00 ± 0.04	15.9	0.126
64	141.4 ± 4.5	0.98 ± 0.04	63.0	0.032
256	142.7 ± 2.3	0.99 ± 0.03	241.9	0.008
none	144.0 ± 4.5	1	356.1	0.006

Batching of system calls. Table 6 isolates the effect of batching system calls, using one worker of the C reactor. When we cap it at one completion per call, `uringpy` makes 2.001 system calls per request, about as many as an `epoll` loop, and delivers 0.80 ± 0.03 of its uncapped throughput. A cap of 4 gives 0.93 ± 0.04 and a cap of 16 gives 1.00 ± 0.04, even though system calls per request keep falling beyond that point, from 0.126 down to 0.006. For a single worker, then, the system calls saved by batching are worth roughly a fifth of its throughput, and almost all of that has been collected by the time a batch holds 16 completions. In practice batches are much larger. An uncapped single worker sees 356 completions per call, and each of four workers, which share the 400 connections, sees 92. With the cap at one, `io_uring` delivers 115.3 thousand requests per second, against 123.4 for the `epoll` loop (the two figures come from different experiments). Without batching, in other words, the completion interface has no advantage on this workload.

What this means for the original hypothesis. The criterion of Section 5.4 is not met. Its first condition was that moving the loop should change throughput far more than changing the interface. On one worker it does not (1.05–1.15× against 1.14–1.25×). On four workers with `io_uring` it does not either, since the two are worth 1.11× and 1.11×. The condition holds only on `epoll` (4.37×), and the control shows that hand-offs are responsible there and not the language. The second condition, that capping the batch should change throughput only slightly, holds for caps of 16 and above and fails below that. We therefore do not claim that the system call is unimportant, and we do not claim that the loop has to be in C.

Scaling across cores. Fig. 4 and Table 7 compare `uringpy` with `asyncio` and `uvloop` on the transport path. Three things stand out.

The first is that `uringpy` worker threads perform like worker processes. At four workers the ratio between the two is 1.00± < 0.01, so on this path one interpreter with four threads delivers what four interpreters deliver. This is what one would hope for from a path that runs no Python, and the measurement confirms that nothing else in the runtime or the interpreter gets in the way.

The second is that its scaling, $S_4 = 2.33 \pm 0.05$, is limited by the server and not by the runtime. A single worker already keeps $C_1 = 1.18$ cores busy, because some of the kernel's network processing for its connections runs on other cores, and four workers keep 3.91 of the four cores busy. Per busy core, throughput at four workers is $E_4 = 0.70$ of what it is at one. The baselines show the same loss in process mode, where no lock is shared at all: their E_4 values are 0.73, 0.72, **??E:scaling/g/uvloop/process/4??** and **??E:scaling/g/uvloop-proto/process/4??**, and they scale by 2.64, 2.50, **??S:scaling/g/uvloop/process/4??** and **??S:scaling/g/uvloop-proto/process/4??**. The loss therefore belongs to the testbed, where four cores are shared between the workers and the kernel's network stack. Process mode marks what the hardware allows (Section 3), and thread-mode `uringpy` reaches it.

The third is that every baseline loses throughput when it is given more threads. With four worker threads `asyncio` delivers 0.52× and 0.48× of one worker through its two APIs, and `uvloop` delivers **??S:scaling/g/uvloop/thread/4??**× and **??S:scaling/g/uvloop-proto/thread/4??**×, with between **??C:scaling/g/uvloop-proto/thread/4??** and 1.76 of the four cores busy. This is the same pattern we saw for `py-epoll` in Table 4, and the context switches point the same way. The `asyncio-proto` threads make **??csw:scaling/g/asyncio-proto/thread/4??** voluntary context switches per request at four workers, compared with **??csw:scaling/g/asyncio-proto/thread/1??** at one. For `uvloop-proto` the figures are **??csw:scaling/g/uvloop-proto/thread/4??** and **??csw:scaling/g/uvloop-proto/thread/1??**. Having the loop in C is not enough to avoid this. `uvloop`'s loop is libuv, but it calls back into Python for every event, and it scales no better than `asyncio`.

In absolute terms, one `uringpy` worker is **??r:uringpy-over-uvloop-proto/thread/g/1??**× faster than one `uvloop-proto` worker, 1.50× faster than `asyncio-proto` and 2.11× faster than `asyncio`. At four workers the fair comparison is with the baselines run as processes, which is how they have to be run under the GIL. Against those, `uringpy` threads are **??r:uringpy-over-uvloop-proto/process/g/4??**× faster than `uvloop-proto`, 1.40× faster than `asyncio-proto` and 1.83× faster than `asyncio`. Against the baselines' threads, which nobody would deploy, the factors are **??r:uringpy-over-uvloop-proto/thread/g/4??**, 7.29 and 9.55. It is worth remembering that this workload flatters the runtime about as much as any could, since the baselines run Python for every event and `uringpy` runs none.

Free-threaded build. The right-hand panel of Fig. 4 and the lower part of Table 7 repeat the experiment without the GIL. `asyncio` threads now scale, by 2.64 ± 0.08 and 2.50 ± 0.05, and reach 0.97 and 0.97 of the through-

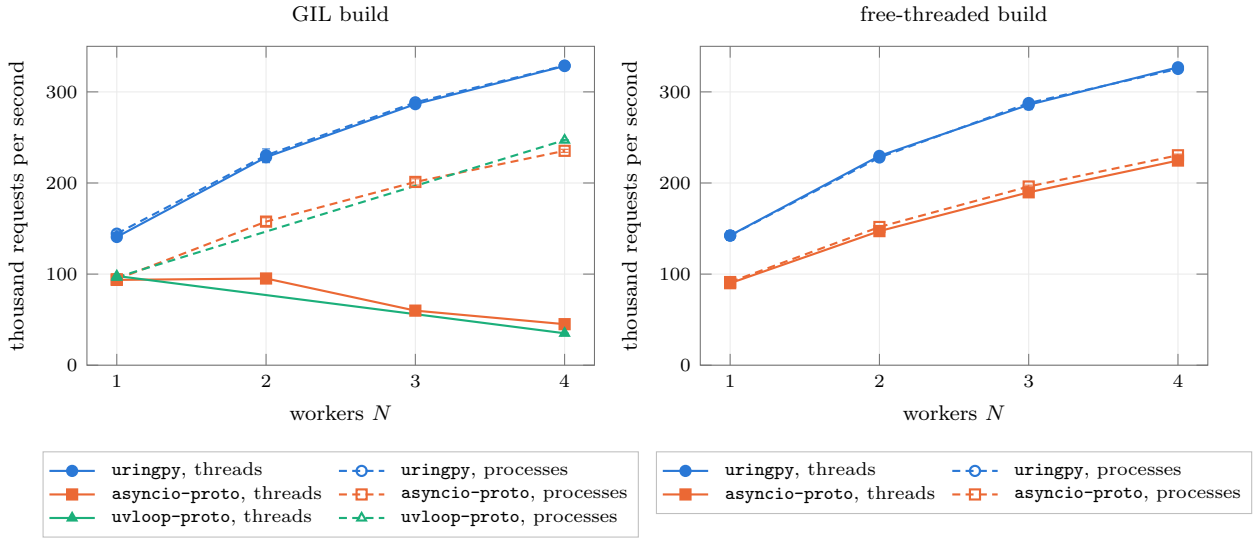


Figure 4: Transport path: throughput against the number of workers, as threads of one interpreter (solid lines, filled markers) and as one process per worker (dashed lines, open markers). Bars are 95% half-widths; most are smaller than the markers. `uvloop` is not available on the free-threaded build.

put of the same engines in process mode. The collapse on the GIL build is therefore caused by the GIL and not by anything in `asyncio`'s design, and it goes away when the GIL does. On one worker the free-threaded build runs `asyncio` at 0.95 and 0.96 of its speed on the GIL build. `uringpy` is unaffected by the choice of build, as one would expect of a loop that does not enter the interpreter. Its throughput at four workers on the two builds differs by a factor of 1.00 ± 0.01. What is left of its advantage is a difference in raw speed: 1.58× over `asyncio-proto` and 2.25× over `asyncio` on one worker, and 1.46× and 1.96× on four. On the free-threaded build, then, the runtime contributes speed per worker and no longer contributes scaling.

A Python handler: one hand-off per request. In Table 8 every request calls a Python function that parses the request line, computes a checksum of the path and builds the response. On one worker the handler leaves `uringpy` with 0.90 of the transport-only throughput in Table 7. When the GIL is taken once per request (`uringpy-app`), worker threads stop scaling after the second one: $S_2 = 1.53 \pm 0.02$ and $S_4 = 1.55 \pm 0.04$, with 3.24 of the four cores busy. Worker processes scale as they do on the transport path ($S_4 = 2.33 \pm 0.06$), so at four workers the threads deliver 0.67 ± 0.02 of what the processes do.

The bound (1) does not explain this, but the handler sweep in Table 9 does. On one worker the handler holds the GIL for $t_p = 1.20$ μs per request, which gives $f = 0.15$. The bound $1/f$ would allow more scaling than four cores can provide. At four threads, however, a request holds the GIL for 2.21 μs and waits 8.51 μs to get it. The GIL utilisation is only $U_4 = 0.41$. For the rest of each second the lock is not running a handler at all, and yet workers are waiting for it. This waiting accounts for the gap between threads and processes. Per worker, a request takes 21.6 μs in thread mode and 13.8 μs in process mode.

Using (2) with $g = 1$, we can estimate the cost of a hand-off as $\tau = 1/\bar{x}_4 - t_{p,4}$ wherever the GIL is the limiting factor. Across the six handler sizes of Table 9 this comes out between 2.8 and 5.6 μs. A hand-off therefore costs several microseconds regardless of what the handler does, and when the handler itself takes only a microsecond or two, the lock is occupied mostly by the hand-off. The spread in these estimates also illustrates the limitation we noted in Section 3, namely that no single value of τ fits all six sizes.

[[PENDING: `tab_gilbatch`: no data yet]]

GIL batching: one hand-off per batch. If the reactor takes the GIL once for all the requests it finds in a pass over the completion queue, f stays the same and g drops from 1 to 0.022. In Table 8 the threads then scale by 2.12 ± 0.07, up from 1.55 ± 0.04. Four threads deliver 275.6 thousand requests per second, compared with 196.3 before (1.40 ± 0.04). That is 0.93 ± 0.02 of what four processes deliver, and a single worker is not slowed down (1.03 ± 0.03). In Table 9 the wait per request at four threads falls from 8.51 to 1.60 μs.

[[PENDING: Check this paragraph against the data of the cap sweep.]] Table **??** varies g in steps while leaving everything else alone, by capping the number of requests handled under one acquisition. These are the connected points in Fig. 3. With a cap of one, the batching code takes the GIL for every request, just as the plain reactor does, and it scales the same way (**??Spm:gilbatch/g/uringpy-app-batch1/thread/4??** against **??Spm:gilbatch/g/uringpy-app/thread/4??**). The batching code path therefore costs nothing in itself. As the cap goes up through 2, 4, 8 and 16, the acquisitions per request fall to **??ho:gilbatch/g/uringpy-app-**

Table 7: Transport path, 13-byte body: throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$) at N workers; scaling $S_4 = \bar{x}_4/\bar{x}_1$; busy server cores C_N; throughput per busy core at four workers relative to one, $E_4 = (\bar{x}_4/C_4)/(\bar{x}_1/C_1)$; 99th-percentile latency at four workers.

Build	Engine	Workers as	N=1	N=2	N=3	N=4	S_4	C_1	C_4	E_4	p99$_4$ (ms)
GIL	`uringpy`	thread	140.7 ± 3.0	228.3 ± 6.0	286.5 ± 3.9	328.4 ± 0.8	2.33 ± 0.05	1.18	3.91	0.70	1.7
GIL	`uringpy`	process	144.4 ± 3.8	230.4 ± 7.0	288.5 ± 1.9	328.9 ± 1.2	2.28 ± 0.06	1.19	3.92	0.69	1.6
GIL	`asyncio`	thread	66.8 ± 1.1	47.8 ± 0.8	35.5 ± 0.1	34.4 ± 0.2	0.52 ± 0.01	1.07	1.76	0.31	15.4
GIL	`asyncio`	process	68.1 ± 1.0	116.8 ± 1.4	151.7 ± 2.7	179.6 ± 0.8	2.64 ± 0.04	1.08	3.91	0.73	2.7
GIL	`asyncio-proto`	thread	93.6 ± 1.6	95.2 ± 1.6	59.9 ± 0.5	45.0 ± 0.9	0.48 ± 0.01	1.12	2.17	0.25	16.7
GIL	`asyncio-proto`	process	94.3 ± 1.6	157.6 ± 4.2	201.2 ± 3.4	235.3 ± 1.7	2.50 ± 0.05	1.12	3.90	0.72	3.5
GIL	`uvloop`	thread	72.5 ± 1.0	–	–	43.0 ± 0.5	0.59 ± 0.01	1.09	1.69	0.38	19.9
GIL	`uvloop`	process	72.4 ± 1.9	–	–	193.6 ± 1.9	2.67 ± 0.07	1.09	3.92	0.74	4.2
GIL	`uvloop-proto`	thread	98.0 ± 2.3	–	–	35.0 ± 0.2	0.36 ± 0.01	1.13	1.59	0.25	26.6
GIL	`uvloop-proto`	process	96.5 ± 1.8	–	–	247.0 ± 0.6	2.56 ± 0.05	1.12	3.90	0.74	3.4
free-thr.	`uringpy`	thread	142.4 ± 5.1	229.7 ± 3.7	285.8 ± 5.3	327.0 ± 1.5	2.30 ± 0.08	1.19	3.91	0.70	1.8
free-thr.	`uringpy`	process	142.2 ± 5.1	228.0 ± 3.6	287.8 ± 2.3	325.2 ± 1.6	2.29 ± 0.08	1.19	3.92	0.69	1.7
free-thr.	`asyncio`	thread	63.2 ± 1.8	108.8 ± 0.6	141.0 ± 2.5	166.6 ± 0.9	2.64 ± 0.08	1.08	3.93	0.73	2.8
free-thr.	`asyncio`	process	64.9 ± 1.1	111.0 ± 1.6	146.1 ± 2.4	172.5 ± 2.6	2.66 ± 0.06	1.08	3.92	0.73	2.8
free-thr.	`asyncio-proto`	thread	90.0 ± 1.8	147.2 ± 2.8	189.8 ± 1.7	224.6 ± 1.2	2.50 ± 0.05	1.12	3.90	0.72	3.7
free-thr.	`asyncio-proto`	process	91.3 ± 1.1	151.7 ± 1.7	196.1 ± 1.5	230.4 ± 1.3	2.52 ± 0.03	1.12	3.92	0.72	3.6

`uvloop` is not installed in the free-threaded image. It was measured in a separate experiment at one and four workers only.

??ho:gilbatch/g/uringpy-app-batch2/thread/4??, **??ho:gilbatch/g/uringpy-app-batch4/thread/4??**, **??ho:gilbatch/g/uringpy-app-batch8/thread/4??** and **??ho:gilbatch/g/uringpy-app-batch16/thread/4??**, and scaling rises to **??S:gilbatch/g/uringpy-app-batch2/thread/4??**, **??S:gilbatch/g/uringpy-app-batch4/thread/4??**, **??S:gilbatch/g/uringpy-app-batch8/thread/4??** and **??S:gilbatch/g/uringpy-app-batch16/thread/4??**. With no cap it is **??Spm:gilbatch/g/uringpy-app-batch/thread/4??**. The wait for the lock and the context switches per request fall in step. This is the relationship between g and scaling that (2) predicts when f is fixed.

[[PENDING: Check this paragraph against the data of the `epoll` reactor.]] GIL batching does not depend on `io_uring`. The `epoll` reactor with the same handler scales by **??Spm:app/g/c-epoll-app/thread/4??** when it takes the GIL per request, and by **??Spm:app/g/c-epoll-app-batch/thread/4??** when it takes it once per pass over the ready sockets, a gain of **??rpm:gilbatch-gain-epoll/g/4??** in throughput at four workers. With batching enabled in both, the `io_uring` reactor is **??rpm:app-iface-batch/g/1??** times faster than the `epoll` reactor on one worker and **??rpm:app-iface-batch/g/4??** times faster on four. As on the transport path, the interface adds speed per worker, while the scaling comes from how the GIL is handled.

Longer handlers: where the lock itself is the limit. As the handler gets longer, f approaches 1 and the bound (1) starts to bind (Fig. 5a). With 300 added iterations ($t_p = 30.75$ µs, $f = 0.82$) the bound is 1.23 and we measure $S_4 = 1.04 \pm 0.03$. With 3000 iterations the bound is 1.03 and the measurement is 1.00 ± 0.01. Batching helps for as long as hand-offs are what keeps the lock busy, and stops helping once the handler takes over that role. With 30 added iterations it raises S_4 from 1.00 to 1.63 and the GIL utilisation from 0.50 to 0.92. With 100 iterations it raises S_4 from 1.11 to 1.24, and from 300 iterations upward it makes no difference. Batching thus gives back what the hand-offs cost, and what remains is the capacity of the lock. It has a price of its own: no response from a batch is queued until all of its handlers have run, and with long handlers the interpreter's switch interval interrupts a batch part-way through. Processes behave in the opposite way and scale better as the handler grows, up to 3.98×. A larger share of each request is then processor time inside the worker, and a smaller share is kernel network processing.

[[PENDING: Check this paragraph against the data of the handler sweep.]] Fig. 5b shows what this means in requests per second. The process-per-worker baselines in it were measured in the same experiment. With the unmodified handler, batched `uringpy` threads deliver 0.96 ± 0.01 of what `uringpy` processes deliver and **??rpm:uringpy-app-batch/thread-over-uvloop-proto-app/pr** of what the strongest baseline, `uvloop` processes, delivers. With 30 added iterations these two ratios become 0.65 ± 0.01 and **??rpm:uringpy-app-batch/thread-over-uvloop-proto-app/pr** With 100 they are 0.42 ± 0.03 and **??rpm:uringpy-app-batch/thread-over-uvloop-proto-app/pr** and with 300 they are 0.30 ± 0.01 and **??rpm:uringpy-app-batch/thread-over-uvloop-proto-app/pr** Worker threads under the GIL can therefore compete with a process per worker only when the handler holds the GIL for a few microseconds. For anything longer, the runtime's own process mode, or indeed any process-per-worker server, is the faster choice on this build.

Baselines with the handler. Given the same handler, the baselines' threads drop to 0.52× (`asyncio-app`), 0.48× (`asyncio-proto-app`) and 0.35×

Table 8: Application-handler workload: every request calls a Python handler. Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$), scaling, and at four workers: GIL acquisitions per request where the reactor counts them and the workers share one GIL, busy server cores and 99th-percentile latency.

Build	Engine	Workers as	N=1	N=2	N=4	S_2	S_4	acq./req	C_4	p99$_4$ (ms)
GIL	`uringpy-app`	thread	126.3 ± 1.7	192.7 ± 1.6	196.3 ± 4.5	1.53 ± 0.02	1.55 ± 0.04	1.000	3.24	3.5
GIL	`uringpy-app`	process	125.8 ± 1.8	202.5 ± 3.5	292.7 ± 5.8	1.61 ± 0.04	2.33 ± 0.06	–	3.90	1.9
GIL	`uringpy-app-batch`	thread	130.1 ± 3.5	204.1 ± 2.6	275.6 ± 5.6	1.57 ± 0.05	2.12 ± 0.07	0.022	3.70	2.1
GIL	`uringpy-app-batch`	process	128.9 ± 2.7	206.4 ± 1.7	297.4 ± 0.8	1.60 ± 0.04	2.31 ± 0.05	–	3.92	1.8
GIL	`asyncio-app`	thread	59.9 ± 1.2	40.2 ± 0.4	31.2 ± 0.5	0.67 ± 0.02	0.52 ± 0.01	–	1.69	17.1
GIL	`asyncio-app`	process	60.2 ± 1.8	104.7 ± 1.3	158.7 ± 9.5	1.74 ± 0.06	2.64 ± 0.18	–	3.93	3.4
GIL	`asyncio-proto-app`	thread	80.7 ± 0.8	64.1 ± 0.9	38.6 ± 0.4	0.79 ± 0.01	0.48 ± 0.01	–	1.91	18.8
GIL	`asyncio-proto-app`	process	81.0 ± 1.7	137.5 ± 1.9	187.1 ± 21.0	1.70 ± 0.04	2.31 ± 0.26†	–	3.92	5.8
GIL	`uvloop-proto-app`	thread	84.6 ± 2.1	68.9 ± 0.7	29.9 ± 0.3	0.81 ± 0.02	0.35 ± 0.01	–	1.46	33.3
GIL	`uvloop-proto-app`	process	85.2 ± 1.5	143.2 ± 2.4	219.9 ± 1.5	1.68 ± 0.04	2.58 ± 0.05	–	3.92	3.7
free-thr.	`uringpy-app`	thread	125.3 ± 1.5	199.2 ± 6.4	290.5 ± 2.3	1.59 ± 0.05	2.32 ± 0.03	–	3.91	1.9
free-thr.	`uringpy-app`	process	125.8 ± 1.8	202.4 ± 2.5	293.8 ± 1.9	1.61 ± 0.03	2.34 ± 0.04	–	3.91	1.8
free-thr.	`uringpy-app-batch`	thread	127.1 ± 3.1	204.1 ± 7.9	290.9 ± 6.8	1.61 ± 0.07	2.29 ± 0.08	–	3.90	2.0
free-thr.	`uringpy-app-batch`	process	129.6 ± 4.4	208.6 ± 3.6	296.4 ± 1.6	1.61 ± 0.06	2.29 ± 0.08	–	3.91	1.8
free-thr.	`asyncio-app`	thread	56.5 ± 0.8	96.8 ± 0.6	151.1 ± 2.3	1.71 ± 0.03	2.68 ± 0.05	–	3.93	3.2
free-thr.	`asyncio-app`	process	56.9 ± 0.5	98.8 ± 2.9	155.2 ± 2.2	1.74 ± 0.05	2.73 ± 0.05	–	3.91	3.1
free-thr.	`asyncio-proto-app`	thread	78.6 ± 1.0	129.2 ± 0.4	198.3 ± 1.2	1.64 ± 0.02	2.52 ± 0.04	–	3.92	4.2
free-thr.	`asyncio-proto-app`	process	79.4 ± 1.7	134.2 ± 1.8	204.5 ± 3.2	1.69 ± 0.04	2.58 ± 0.07	–	3.90	4.0

† A configuration in this row had a coefficient of variation above 5%.

(`uvloop-proto-app`) of one worker. The strongest baseline is `uvloop-proto-app` in process mode, which reaches 219.9 thousand requests per second on four workers. Measured against it, `uringpy-app` threads deliver 0.89 ± 0.02 when the GIL is taken per request and 1.25 ± 0.03 with GIL batching. Process against process, `uringpy-app` delivers 1.33 ± 0.03. On one worker `uringpy-app` is 1.49× faster than `uvloop-proto-app`, 1.56× faster than `asyncio-proto-app` and 2.11× faster than `asyncio-app`.

Free-threaded build with the handler. Without the GIL there are no hand-offs to save. `uringpy-app` threads scale by 2.32 ± 0.03 and deliver 0.99 ± 0.01 of what its processes do, and GIL batching makes no difference (1.00 ± 0.02). `asyncio-proto-app` threads scale by 2.52 ± 0.04, and the ratio between the two engines at four workers is 1.46 ± 0.01. Batched threads on the GIL build reach 0.95 of the throughput of threads on the free-threaded build.

The free-threaded series in Fig. 5a comes with a warning for longer handlers, though. Thread scaling there falls as the handler grows, to 1.20 with 300 added iterations and 1.01 with 3000, which is no better than under the GIL. We ran a control without the runtime and saw the same thing. Plain Python threads calling the handler with 300 iterations complete **??control:f/h300/1??**, **??control:f/h300/2??** and **??control:f/h300/4??** thousand calls per second with one, two and four threads on the free-threaded build, and **??control:g/h300/1??**, **??control:g/h300/2??** and **??control:g/h300/4??** on the GIL build. The effect therefore comes from this interpreter version running this particular handler, an interpreted integer loop, and has nothing to do with the runtime. We did not track down its cause, and it should not be taken as a general statement about the free-threaded build. It does mean that the free-threaded results in Table 8 apply to the short handler measured there and may not carry over to longer ones.

[[PENDING: `tab_load`: no data yet]]

[[PENDING: Check this paragraph against the data of the connection sweep.]] **Load.** Everything so far was measured with 400 connections. Batch sizes, and with them the number of hand-offs per request, depend on how many requests are waiting when a worker makes a pass, so in Table **??** we vary the number of connections from 16 to 1600. With 16 connections each of four workers has only four, and batches are small. `uringpy-app-batch` then takes the GIL **??ho:load/g/uringpy-app-batch/thread/4/c16??** times per request, compared with **??ho:load/g/uringpy-app-batch/thread/4??** at 400 connections, and scales by **??Spm:load/g/uringpy-app-batch/thread/4/c16??**, where the plain reactor scales by **??Spm:load/g/uringpy-app/thread/4/c16??**. The Python loop on `epoll` scales by **??Spm:load/g/py-epoll/thread/4/c16??**, **??Spm:load/g/py-epoll/thread/4/c64??**, **??Spm:load/g/py-epoll/thread/4??** and **??Spm:load/g/py-epoll/thread/4/c1600??** at 16, 64, 400 and 1600 connections, and the Python loop on `io_uring` by **??Spm:load/g/py-uring/thread/4/c16??**, **??Spm:load/g/py-uring/thread/4/c64??**, **??Spm:load/g/py-uring/thread/4??** and **??Spm:load/g/py-uring/thread/4/c1600??**. The benefit of batching hand-offs therefore grows with load and is smallest when few requests are pending.

Response size. In Table 10 we vary the size of the response body, using four worker processes on the GIL build

Table 9: Handler work swept (iterations of an interpreted loop added to every request), GIL build unless stated, with the GIL taken once per request (`uringpy-app`) or once per batch of requests (`uringpy-app-batch`). One-worker throughput $\bar{x}_1$ in thousands of requests per second (mean ± 95% half-width, $n = 5$); GIL hold time per request t_p at one worker; $f = t_p\,\bar{x}_1$; the bound $\min(4, 1/f)$ of (1); measured scaling of worker threads (S_2, S_4), of worker processes (S_4^{proc}) and of threads on the free-threaded build (S_4^{ft}); and, at four worker threads, GIL hold time $t_{p,4}$ and wait time w_4 per request, GIL acquisitions per request, and GIL utilisation $U_4 = \bar{x}_4\,t_{p,4}$.

GIL per	Work	$\bar{x}_1$	t_p (μs)	f	bound	S_2	S_4	S_4^{proc}	S_4^{ft}	$t_{p,4}$	w_4	acq.	U_4
request	0	123.2 ± 1.8	1.20	0.15	4.00	1.54 ± 0.05	1.50 ± 0.03	2.36 ± 0.13	2.12 ± 0.43†	2.21	8.51	1.000	0.41
request	30	91.3 ± 1.1	4.11	0.38	2.66	1.46 ± 0.04	1.00 ± 0.02	2.64 ± 0.03	2.14 ± 0.34†	5.42	26.36	1.000	0.50
request	100	55.4 ± 1.3	11.09	0.62	1.63	1.23 ± 0.05	1.11 ± 0.03	3.08 ± 0.08	1.70 ± 0.04	12.61	43.09	1.000	0.77
request	300	26.5 ± 0.2	30.75	0.82	1.23	1.05 ± 0.05	1.04 ± 0.03	3.51 ± 0.03	1.20 ± 0.02	33.38	103.26	1.000	0.92
request	1000	8.62 ± 0.38	106.23	0.92	1.09	1.05 ± 0.05	0.98 ± 0.05	3.92 ± 0.03	1.07 ± 0.07	113.08	348.42	1.000	0.95
request	3000	3.07 ± 0.03	315.80	0.97	1.03	0.98 ± 0.08†	1.00 ± 0.01	3.98 ± 0.07	1.01 ± 0.07†	320.40	969.16	1.000	0.98
batch	0	126.6 ± 2.0	1.24	0.16	4.00	1.61 ± 0.04	2.20 ± 0.04	–	–	2.42	1.60	0.021	0.67
batch	30	96.2 ± 0.9	4.00	0.38	2.60	1.54 ± 0.07	1.63 ± 0.03	–	–	5.92	12.27	0.021	0.92
batch	100	57.8 ± 0.2	10.87	0.63	1.59	1.33 ± 0.02	1.24 ± 0.08†	–	–	13.44	36.70	0.020	0.96
batch	300	27.1 ± 0.4	30.51	0.83	1.21	1.09 ± 0.05	1.04 ± 0.04	–	–	35.02	102.09	0.020	0.98
batch	1000	8.99 ± 0.12	102.44	0.92	1.09	1.02 ± 0.06	1.00 ± 0.02	–	–	212.83	222.60	0.021	n/a
batch	3000	3.14 ± 0.07	310.27	0.97	1.03	1.00 ± 0.02	0.98 ± 0.02	–	–	978.66	306.73	0.022	n/a

$t_{p,4}$ and w_4 in μs. The per-request and per-batch rows come from separate experiments. n/a: with batching and long handlers the interpreter's switch interval interrupts a batch, so the recorded hold includes time waiting to resume. † A configuration in this ratio had a coefficient of variation above 5%.

Table 10: Response-body size (four worker processes, GIL build). Throughput in thousands of requests per second (mean ± 95% half-width, $n = 5$); for `uringpy` also the data rate received by the client, busy server cores and system calls per request.

Body	`uringpy`	`asyncio-proto`	Gbit/s	C	sys/req
64 B	327.9 ± 0.9	234.9 ± 1.6	0.4	3.91	0.02
1 KiB	321.0 ± 1.6	230.7 ± 1.5	2.9	3.92	0.02
16 KiB	119.3 ± 0.5	118.9 ± 0.7	15.7	3.32	0.64
64 KiB	30.0 ± 0.1	30.0 ± 0.1	15.8	1.45	0.74
256 KiB	7.49 ± 0.04	7.50 ± 0.01	15.8	1.29	1.23
1 MiB	1.87 ± <0.01	1.87 ± 0.01	15.8	1.61	2.44

and comparing against `asyncio`'s Protocol API. With a 64-byte body `uringpy` is 1.40× faster, and with 1 KiB it is 1.39× faster. At 16 KiB the ratio is 1.00, and beyond that the two engines cannot be told apart. The client receives between 15.7 and 15.8 Gbit/s whatever the body size and whichever engine is serving, which tells us that the network path has become the bottleneck and those rows no longer measure the servers. As responses grow, each one is sent in several pieces and batches shrink, so the system calls `uringpy` makes per request rise from 0.023 to 2.443. The runtime's advantage in requests per second is thus limited to responses small enough for per-request overhead to dominate over data transfer, which on this network means below 16 KiB. We did not investigate what limits the network path.

6 Discussion

Attributing a speed-up. If a Python loop on `io_uring` is faster than `asyncio`, that does not by itself show that `io_uring` is the reason. On one worker the interface is worth a factor of 1.14–1.25, the saving from fewer system calls is almost used up at 16 completions per call, and a bare C loop is only 1.05–1.15× faster than the same loop in Python. A figure of several hundred completions per system call is real, but it says little about single-worker throughput. Most of the distance between one `asyncio` worker and one `uringpy` worker (2.11×) comes from the work a full event loop does for each event. The interface it waits on and the language its dispatch is written in account for much less.

What batching buys. For worker threads under the GIL, the valuable thing about batching is that the lock is given up once per batch, where it would otherwise be given up at every system call or every request. A completion interface provides this automatically, and that is why loops built on `io_uring` can scale across threads where the usual readiness loop collapses. The benefit belongs to the batching of hand-offs, however, and the interface is only one way to get it. An `epoll` loop that batches its releases scales just as well, while an `io_uring` loop that holds the GIL across its submitting call does not scale at all. An `asyncio`-compatible loop on `io_uring` could fall on either side, depending on where it releases the GIL and how much Python it runs per event. We did not measure the existing ones.

Three ways to keep hand-offs rare. The runtime offers a reactor that runs no Python, a reactor that calls a handler under one GIL acquisition per batch, and a batch interface for loops written in Python. On the transport path the first is 1.11× faster than the third at four workers, so its advantage over a well-behaved Python loop is modest. With a short handler, the second makes a larger difference. Per-request locking delivers 0.89 of the best process-per-worker baseline, and per-batch locking delivers 1.25 of it, still inside a single interpreter.

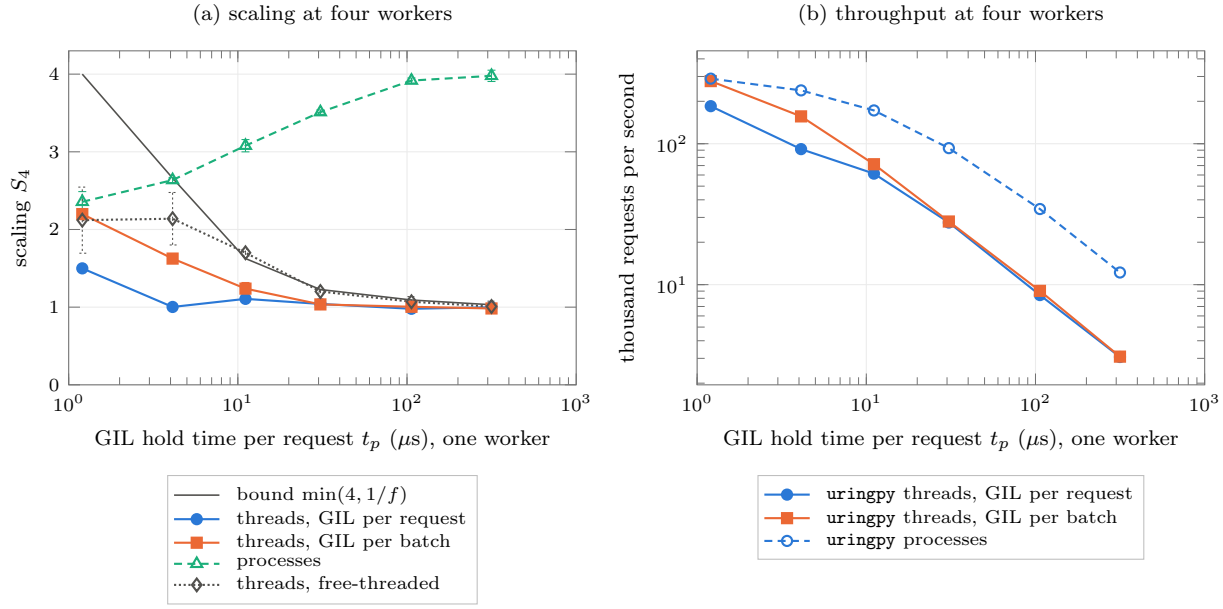


Figure 5: Handler cost swept (GIL build unless stated). The horizontal axis is the measured time one request holds the GIL on one worker. (a) Scaling at four workers against the bound of (1); bars are 95% half-widths. (b) Throughput at four workers: worker threads of one interpreter (solid) against one process per worker (dashed), for `uringpy` and for the two baselines through the Protocol API.

The limit that remains. Hand-offs are an overhead that can be designed away. The time the handler itself holds the GIL cannot. Once a request holds the GIL for more than a few microseconds, threads under the GIL fall behind a process per worker whatever the design (Fig. 5b), and one needs processes or the free-threaded build. On the free-threaded build the runtime keeps an advantage in speed per worker for short handlers. Our one longer handler did not scale there either, for reasons that lie in the interpreter.

Threads or processes. Worker processes reach at least the throughput of `uringpy`'s worker threads, so speed is not an argument for threads. The argument is that the workers live in one interpreter and can share state without inter-process communication. We did not measure a workload that depends on shared state. What this paper establishes is that the option exists at little or no cost in throughput, for the transport path and for short handlers. How much a particular application would gain from it remains open.

7 Threats to Validity

Table 11 lists the ways in which a measurement of this kind can mislead, and what we did about each. The limitations that remain are discussed below.

Testbed. The server has four cores, so we cannot show scaling beyond four workers, and at four workers the kernel's network processing competes with the workers for the same cores (E_4 in Table 7 quantifies this). We do not know whether the ordering in Fig. 3 holds with more cores, where more threads compete for the GIL, although the model gives no reason to expect hand-offs to get cheaper. The machines are cloud virtual machines of one family, running one kernel and one CPython version (3.14.7). All runs were made on one pair of machines within a single day. The free-threaded results describe that version of an implementation that is still changing. Above 16 KiB responses the network path was the bottleneck.

Workload. The workloads consist of one protocol exchange with a fixed or lightly computed response. Requests are not parsed, and we do not cover TLS, request bodies or realistic application logic. The transport workload runs no Python in `uringpy` and runs Python for every event in the baselines, which is the most favourable case for the runtime. Load comes from a closed-loop generator over keep-alive connections. Table **??** varies their number but not the arrival pattern, and we did not measure open-loop or bursty load. We use `io_uring` in its basic form (Section 4) and did not evaluate features such as multishot receive or deferred task running. The Python loops of the 2×2 design are bare, so the loop-placement effect on one worker is a lower bound for a full event loop. The handler is one small function, optionally lengthened by an interpreted loop. Handlers that release the GIL themselves, for I/O or inside native code, were not measured and would behave differently.

The model. The data support (2) as an ordering and not as a predictor. We derive the hand-off cost τ from throughput and hold time and do not time it directly, and no single value fits every configuration: it ranges from 2.8 to 5.6 μs across handler sizes. The hand-off control

changes f as well as g (Section 5.5). We count GIL hand-offs for our own engines only. For `asyncio` and `uvloop` we have context switches, divided by an estimated request count. The batch cap for system calls was varied only for the C reactor on one worker.

What was not measured. We did not run the `io_uring` event loops that motivated the study, and so we make no claim about them. We reproduced the slowdown of the interpreted loop across threads on the free-threaded build without the runtime, but did not explain it. We measured no workload that benefits from state shared between workers.

Measurement. Latency comes from a closed-loop generator and is not an estimate of tail latency; the latency columns are means of per-run 99th percentiles. The contention probe is usable only with one competing thread on this machine (Section 3). Runs with the longest handlers reported socket errors (Section 5.5). The `io_uring` reactor accepts one connection per pass, so when a pass takes a long time the client's 400 connections are established slowly. We believe the errors are connection attempts that timed out during that ramp-up, but we did not verify this. They affect at most 0.39% of a run's requests. 7 of the 227 configurations had a coefficient of variation above 5%, and those that enter a reported ratio are marked in the tables. One test in the runtime's test suite failed once on the free-threaded image in an earlier revision and has passed in every run since. We did not find the cause.

Comparisons across experiments. Repetitions are interleaved within an experiment but not between experiments. One `uringpy` worker was measured in three of them, at 140.7 ± 3.0, 140.3 ± 4.3 and 144.0 ± 4.5 thousand requests per second. Ratios between configurations of the same experiment are not affected by this. Comparisons that span two experiments carry the additional uncertainty, and where we give intervals for them they reflect only the variation within each experiment. This applies to the capped `io_uring` loop against `epoll`, to `py-epoll` against `asyncio`, to the cost of the handler on one worker, and to every comparison between the GIL build and the free-threaded build.

Hypothesis and analysis. We formulated the central statement after seeing the first data (Section 5.4). Of the experiments that test it, one has its prediction on record from before the run. Apart from that, the analysis did not follow a pre-registered plan. The earlier measurements, including those that led us to a wrong conclusion, are kept in the repository.

8 Conclusion

What batching buys a Python server depends on how many threads share the interpreter. For a single worker it saves system calls. Together with the interface this is worth a factor of 1.14–1.25, and most of it has been collected by the time a batch holds 16 completions. For several worker threads under the GIL, batching does something more useful: the lock is handed over once per batch, where it would otherwise change hands at every system call or every request. In our measurements this is what separates designs that scale from designs that lose throughput, and neither the interface nor the language of the loop does. A Python loop on `epoll` scales by **??S:factorial/g/py-epoll-batch/thread/4??**× on four cores if it releases the GIL once per pass, and falls to 0.64× if it releases it at every call, with the interface and the system calls unchanged. A Python loop on `io_uring` scales by 2.21×, and falls to 0.98× if it merely holds the GIL during its submitting call.

A runtime built on this observation lets one interpreter on a standard build match a process per worker when no Python runs per request. With a short Python handler, taking the GIL once per batch brings it to 0.93 of the process throughput. The approach has limits, and we measured them. Once the handler holds the GIL for more than a few microseconds, a process per worker is faster, and no design restores thread scaling under the GIL. On the free-threaded build `asyncio` threads scale as well, and what is left is a difference in speed per worker. With responses of 16 KiB or more, the network was the bottleneck in our setup. For a multi-threaded Python server, then, the cost that batching most needs to spread out is the GIL hand-off, and the system call comes second.

Several questions remain open. We measured on four cores, and it would be worth knowing how the cost of a hand-off behaves when more threads compete for the lock. We did not run the existing `io_uring` event loops, and a study of where each of them releases the GIL would show which regime it is in. Finally, our handlers were synthetic. A server that parses HTTP in C and enters Python only for application code is the natural next step, and it would show how much of a realistic request can be kept outside the lock.

Acknowledgment

[[PENDING: Author to confirm the wording before submission. IEEE requires content generated by AI to be disclosed in this section.]] The author used Claude (Anthropic), an AI assistant, in revising this work: to review an earlier draft and propose additional experiments in response; to write parts of the implementation (the application-handler reactor, GIL batching, the `epoll` reactor and its batch helper, the control loops, the benchmark driver, the generator of tables, figures and quoted numbers, and the tests) and of the documentation; and to draft text in every section of this paper, including the abstract. The author ran all experiments, reviewed and tested this content, and is responsible for the paper's claims.

Table 11: Pitfalls and how each is handled.

Pitfall	What is done	Evidence
The load generator, not the server, is the limit	The client has twice the server's cores; client processor use is recorded for every run and configurations above 85% are flagged	Section 5.5; raw data
Hyper-threads counted as cores	Machine family with one vCPU per physical core	Table 2
Noise mistaken for an effect	Five repetitions, interleaved in shuffled order, a fresh server per run, 95% confidence intervals; cross-experiment spread stated	every results table
A weak baseline	`asyncio` through both its streams and its faster Protocol API, and `uvloop`, with and without the handler; process mode as well as thread mode, also in the handler sweep; one interpreter version, one listen backlog and `TCP_NODELAY` for every engine	Tables 7, 8; Fig. 5
GIL and free-threaded builds differing in more than the GIL	Both interpreters built from one recipe; the GIL state reported by the server is recorded per run	Table 2
Crediting `io_uring` for what C code does, or the reverse	The 2×2 design varies the two separately; the batch cap varies batching alone	Tables 4–6
Crediting the interface or the loop's language for what the handling of the GIL does	Two controls that each change one thing: GIL releases with interface and system calls fixed; time under the GIL with releases fixed. Releases, acquisitions and context switches counted. The same batching applied to an `epoll` reactor	Tables 4, 8; Fig. 3
A correlation across different engines read as a cause	One engine with the number of requests per GIL acquisition varied in steps and nothing else changed	Table ??
System-call counts estimated, not measured	Every engine of ours counts its system calls and completed requests	Tables 4, 6
A one-worker baseline that also uses other cores	Server processor use recorded; busy cores and throughput per busy core reported next to scaling	Table 7
A model fed with assumed numbers	GIL hold time, wait time and acquisitions measured per request; the bound compared with measured scaling at six handler sizes	Table 9
Results that hold at one load only	Connections varied from 16 to 1600	Table ??
Engines doing different protocol work	Every engine answers each `recv` with identical bytes; none parses HTTP	Section 4
Errors hidden in a throughput figure	Socket errors, non-2xx responses, handler errors and failed runs recorded and reported	Section 5.5; raw data
Code changing between experiments	All experiments run at one commit by one script, after the test suite has passed in both images	Section 5
Numbers not traceable to code, or mistyped	Commit, image, interpreter and seed recorded per experiment; raw runs archived, superseded runs included; tables, figures and the numbers in the text generated from them	repository

Code and Data Availability

The runtime and all benchmark scripts are open source (MIT) at `https://github.com/marimuthuvm/uringpy` and released on PyPI (`pip install uringpy`). The repository contains the raw output of every run reported here and of the earlier, superseded runs (`benchmarks/results/`), the script that generates the tables, the figures and the numbers quoted in the text from it (`benchmarks/make_tables.py`), the script that ran all experiments at one commit (`benchmarks/run_paper_experiments.sh`), the container recipe for both interpreters, and a description of each experiment (`benchmarks/EXPERIMENTS.md`). **[[PENDING: Archive the release that contains these results and cite its DOI here; the earlier archive,** `https://doi.org/10.5281/zenodo.23129560`**, predates them.]]**

Conflict of Interest

The author has no financial or personal relationships that could have influenced this work, and declares no conflict of interest. No external funding was received.

References

[1] J. Axboe, "Efficient IO with io_uring," 2019. [Online]. Available: `https://kernel.dk/io_uring.pdf`

[2] "uringloop: A liburing-based proactor event loop for asyncio." [Online]. Available: `https://github.com/bright2227/uringloop`

[3] A. K. Pandey, "uringcore: High-performance asyncio event loop for Python using io_uring." [Online]. Available: `https://github.com/ankitkpandey1/uringcore`

[4] L. Soares and M. Stumm, "FlexSC: Flexible system call scheduling with exception-less system calls," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010.

[5] S. Han, S. Marshall, B.-G. Chun, and S. Ratnasamy, "MegaPipe: A new programming interface for scalable network I/O," in *Proc. 10th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2012, pp. 135–148.

[6] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, "Understanding modern storage

APIs: A systematic study of libaio, SPDK, and io_uring," in *Proc. 15th ACM Int. Systems and Storage Conf. (SYSTOR)*, 2022, pp. 120–127, doi: 10.1145/3534056.3534945.

[7] M. Jasny, M. El-Hindi, T. Ziegler, V. Leis, and C. Binnig, "High-performance DBMSs with io_uring: When and how to use it," *Proc. VLDB Endow.*, vol. 19, no. 9, pp. 2317–2330, 2026, doi: 10.14778/3819518.3819553.

[8] D. Beazley, "Understanding the Python GIL," presented at PyCon, Atlanta, GA, USA, 2010. [Online]. Available: `http://www.dabeaz.com/GIL/`

[9] S. Gross, "PEP 703: Making the Global Interpreter Lock Optional in CPython," Python Enhancement Proposals, 2023.

[10] T. Wouters, M. Page, and S. Gross, "PEP 779: Criteria for supported status for free-threaded Python," Python Enhancement Proposals, 2025.

[11] K. Aditya, "Scaling asyncio on free-threaded Python," Quansight Labs, Sep. 2025. [Online]. Available: `https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python`

[12] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith, "Cython: The best of both worlds," *Comput. Sci. Eng.*, vol. 13, no. 2, pp. 31–39, 2011.

[13] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, vol. 30, 1967, pp. 483–485.

[14] S. Eyerman and L. Eeckhout, "Modeling critical sections in Amdahl's law and its implications for multicore design," in *Proc. 37th Annu. Int. Symp. Computer Architecture (ISCA)*, 2010, pp. 362–370, doi: 10.1145/1815961.1816011.

[15] MagicStack, "uvloop: Ultra fast asyncio event loop." [Online]. Available: `https://github.com/MagicStack/uvloop`

[16] "Granian: A Rust HTTP server for Python applications." [Online]. Available: `https://github.com/emmett-framework/granian`

[17] M. Kerrisk, "The SO_REUSEPORT socket option," *LWN.net*, Mar. 2013. [Online]. Available: `https://lwn.net/Articles/542629/`

[18] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007, pp. 57–76, doi: 10.1145/1297027.1297033.

[19] W. Glozer, "wrk: Modern HTTP benchmarking tool." [Online]. Available: `https://github.com/wg/wrk`

[20] Google Cloud, "CPU platforms," Compute Engine documentation. [Online]. Available: `https://docs.cloud.google.com/compute/docs/cpu-platforms`

Marimuthu Velayutham is an independent researcher working on high-performance systems software and language runtimes, with interests in asynchronous I/O, operating-system interfaces, and the performance of managed-language runtimes.